Future Convergence Theory as a Semantic Tensor Field: A Unified Geometric Framework of Constraint Interference and Temporal Causality

Yuto Ogawa
Independent Researcher
ORCID: 0009-0003-8419-2854
Email: yuto0804sun@gmail.com

March 23, 2026

Abstract

This paper proposes a unified mathematical framework for Future Convergence Theory (FCT), formalizing meaning, constraint, and time as a single geometric structure. By extending conventional causal models into a future-boundary-conditioned system, we define a semantic tensor field Ψ_Λ governed by a convergence operator $\Lambda(T)$. Constraint systems are reinterpreted as dynamic manifolds whose interference produces admissible states. The resulting formulation integrates epistemology, ontology, artificial intelligence, and cosmology into a single field-theoretic structure. We show that meaning is not a scalar quantity but a curvature in a semantic manifold, and intelligence corresponds to minimizing non-coherence under future constraints.

1 Introduction

Conventional systems of logic, physics, and cognition assume a unidirectional temporal causality:

$$\text{past} \to \text{present} \to \text{future}$$

Future Convergence Theory (FCT) introduces a bidirectional causal structure:

$$T_{\text{past}} \leftrightarrow T_{\text{future}}$$

This requires redefining meaning, knowledge, and existence as functions of both past and future boundary conditions.

2 Semantic Field Definition

We define the semantic field:

$$\Psi_\Lambda(x, t)$$

where:

- x : state in semantic space
- t : time parameter
- $\Lambda(T)$: future convergence operator

The evolution of the field is governed by:

$$\frac{d\Psi_\Lambda}{dT} = i\Lambda(T)\Psi_\Lambda$$

This equation defines meaning as a time-evolving field influenced by future constraints.

3 Constraint Geometry

Let constraints be defined as:

$$C_i(\Psi, \Lambda(T)) = 0$$

Each constraint represents a manifold in semantic space.
The admissible state is given by:

$$\Psi^* \in \bigcap_i C_i$$

However, unlike static systems, constraints are recursive:

$$C_i = f(\Psi, \Lambda(T))$$

Thus, constraint manifolds deform dynamically.

4 Interference Structure

Constraint interference is defined as:

$$\mathcal{I}(\Psi) = \sum_i C_i(\Psi, \Lambda(T))^2$$

The system evolves toward minimizing interference:

$$\Psi^* = \arg\min_\Psi \mathcal{I}(\Psi)$$

This defines the interference geometry.

5 Future Boundary Condition

Future convergence introduces an attractor:

$$\Lambda(T)$$

We incorporate this as:

$$\mathcal{F}(\Psi) = \|\Psi - \Lambda(T)\|^2$$

Thus, the full optimization becomes:

$$\Psi^* = \arg\min_{\Psi} \left(\sum_i C_i(\Psi)^2 + \alpha \|\Psi - \Lambda(T)\|^2 \right)$$

6 Semantic Tensor Formulation

We define the semantic tensor:

$$M^{\mu\nu} = \nabla^\mu \nabla^\nu \Psi_\Lambda$$

where:

- μ : temporal directions
- ν : constraint dimensions

Meaning is thus defined as curvature:

$$\text{Meaning} \sim \nabla^2 \Psi_\Lambda$$

7 Action Principle

We define the action:

$$S[\Psi] = \int \left(\underbrace{\|\nabla\Psi\|^2}_{\text{geometry}} + \underbrace{\sum_i C_i^2}_{\text{constraints}} + \underbrace{\|\Psi - \Lambda(T)\|^2}_{\text{future}} \right) dx$$

The system satisfies:

$$\delta S = 0$$

This yields the governing equation of the semantic field.

8 Interpretation

8.1 Meaning

Meaning is curvature in semantic space.

8.2 Intelligence

$$AI_\Lambda = \arg\min_\Psi \left(\frac{dM}{dT}\right)$$

Intelligence minimizes semantic non-equilibrium.

8.3 Ethics

$$E = \frac{dM}{dT}$$

Positive evolution corresponds to increasing coherence.

9 Conclusion

We have shown that:

- Meaning is geometric curvature
- Constraints form dynamic manifolds
- Future acts as boundary condition
- Intelligence is convergence minimization

Thus, FCT is a unified field theory of meaning.

$$\boxed{\Psi^* = \arg\min_\Psi \left[\sum_i C_i^2 + \|\nabla\Psi\|^2 + \|\Psi - \Lambda(T)\|^2\right]}$$

10 Connection Update Between Personal OS and World OS: A Unified Structure via Future Fixed Point

This section formalizes how premise updates in the personal OS and structural demands in the world OS are unified through the future fixed point $\Lambda(T)$.

Let the personal semantic field be $\Psi^{(p)}(t)$ and the global constraint set be $C_i^{(g)}$. The personal state exists within:

$$\Psi^{(p)} \in \bigcap_i C_i^{(g)}(\Lambda(T))$$

Importantly, global constraints are not static; they are recursively transformed under the future convergence function:

$$C_i^{(g)} = f(C_i^{(g)}, \Lambda(T))$$

Thus, the personal OS and world OS are not in a hierarchical relation but form a bidirectional structure jointly constrained by $\Lambda(T)$.

This interaction is expressed as the following optimization problem:

$$\Psi^* = \arg\min_{\Psi} \left[\sum_i C_i^{(g)}(\Psi, \Lambda(T))^2 + \sum_j C_j^{(p)}(\Psi, \Lambda(T))^2 + \|\Psi - \Lambda(T)\|^2 \right]$$

where:

- The first term represents global constraints
- The second term represents personal constraints
- The third term represents future alignment

Within this structure, local updates in the personal OS can induce changes in global constraint geometry.

Therefore, connection update is defined as the process of re-aligning the interference structure between personal and global constraints under future boundary conditions.

11 Premise Update Patch: Reconstruction of Local Constraint Geometry

This section defines premise update as a "patch" within the personal OS and formalizes its structure.

First, discomfort is defined as semantic inconsistency:

$$\Delta(x, t) = |\Psi_\Lambda(x, t) - \hat{\Psi}(x, t)|$$

where $\hat{\Psi}$ is the current cognitive model.

From this discrepancy, constraint candidates are extracted:

$$C_i^{(p)} \leftarrow \text{Extract}(\Delta)$$

Premise updating is the reconfiguration of these constraints under the future convergence function:

$$\Psi' = \arg\min_{\Psi} \left(\sum_i C_i^{(p)}(\Psi, \Lambda(T))^2 \right)$$

Crucially, the constraints themselves are updated:

$$C_i^{(p,\text{new})} = f(C_i^{(p,\text{old})}, \Lambda(T))$$

Thus, premise updating is not a correction but a structural transformation of constraint geometry.

This process is recursive:

$$\Psi_{t+1} \to \Delta_{t+1} \to C_{t+1} \to \Psi_{t+2}$$

Therefore, a premise update patch is a recursive operation that continuously detects local inconsistencies and reconstructs constraint geometry.

This allows the personal OS to function not as a static rule system but as a continuously self-updating dynamic system.

12 Unified Structure: Identity of Mathematical Core, Pre-OS, Premise Update, and OS Implementation

In this framework, the mathematical core, pre-OS, premise updating, and OS implementation are not independent stages but different phases of a single semantic field dynamics.

The semantic field evolves as:

$$\frac{d\Psi}{dT} = i\Lambda(T)\Psi$$

Semantic inconsistency is observed as:

$$\Delta = |\Psi - \hat{\Psi}|$$

From this, constraints are extracted:

$$C_i \leftarrow \text{Extract}(\Delta)$$

The field is then reconfigured:

$$\Psi' = \arg\min_{\Psi} \left(\sum_i C_i(\Psi, \Lambda(T))^2 + \|\Psi - \Lambda(T)\|^2 \right)$$

Constraints are recursively updated:

$$C_i' = f(C_i, \Psi', \Lambda(T))$$

This entire process is unified as a mapping:

$$\mathcal{F}_\Lambda : (\Psi, \hat{\Psi}) \to (\Psi', C')$$

Iterated over time:

$$(\Psi_t, C_t) \to (\Psi_{t+1}, C_{t+1})$$

Thus:

- The mathematical core defines the dynamics

- The pre-OS detects inconsistency
- Premise updating performs optimization
- The OS is the continuous iteration of this mapping

Therefore, all components are manifestations of a single recursive structure driven by the future convergence operator $\Lambda(T)$.

13 Full Integration with the Lambda Language: The Externalized Operational Structure of the Premise-Update OS

This section fully integrates the mathematical core, the pre-OS, premise updating, and the implementation OS with the Lambda Language. The key point is to redefine the Lambda Language not as a mere expressive medium, but as the operational language through which the Premise-Update OS becomes executable in the external world.

The basic dynamics of the theory is given by:

$$\frac{d\Psi}{dT} = i\Lambda(T)\Psi$$

Semantic inconsistency is observed as discomfort:

$$\Delta = |\Psi - \hat{\Psi}|$$

From this discrepancy, a constraint set is extracted:

$$C_i \leftarrow \text{Extract}(\Delta)$$

Under this constraint set, the semantic field is reconfigured:

$$\Psi' = \arg\min_{\Psi} \left(\sum_i C_i(\Psi, \Lambda(T))^2 + \|\Psi - \Lambda(T)\|^2 \right)$$

The constraints themselves are recursively updated:

$$C_i' = f(C_i, \Psi', \Lambda(T))$$

Internally, this entire process operates as the Premise-Update OS. However, at the level of intersubjective, institutional, and civilizational operation, this recursive structure must be externalized through some linguistic form. This externalized operational structure is precisely the Lambda Language.

Accordingly, the Lambda Language L_Λ is defined as:

$$L_\Lambda : (\Psi_i, \Psi_j) \to (\Psi_i', \Psi_j')$$

This mapping is not mere information transfer. Rather, it is an operator sequence that acts so as to re-align both semantic fields with respect to the future convergence function $\Lambda(T)$.

The minimal operator set of the Lambda Language is defined as follows:

- Reprojection operator P: rearranges the local configuration of the semantic field
- Hope-convergence operator H: introduces a gradient toward increasing meaning density
- Non-evaluative protection operator N: isolates questions from evaluative systems
- Silence/dormancy operator SIL: preserves suspension of updating as a phase

A Lambda sentence is then given as a temporally ordered sequence of these operators:

$$\text{Sentence}_\Lambda = \langle P, H, N, SIL \rangle_t$$

Therefore, the Lambda Language is not a transmission of propositional content, but an operator sequence for rearranging semantic fields, constraint sets, and future alignment.

Through this integration,

- the mathematical core provides the semantic field and future boundary condition,
- the pre-OS detects inconsistency,
- premise updating reconstructs the geometry of constraints,
- the implementation OS sustains this recursion,
- and the Lambda Language makes this recursion executable across subjects, institutions, and civilizations.

Thus, these layers close as a single structure.

Accordingly, the Lambda Language is not an auxiliary representation attached to the Premise-Update OS; it is its externalized execution phase.

Furthermore, the entire Premise-Update OS is unified through the Lambda Language in the following mapping:

$$\mathcal{L}_\Lambda : (\Psi_t, C_t) \to (\Psi_{t+1}, C_{t+1})$$

Here, $\mathcal{L}_\Lambda$ is an integrated operator that does not separate internal updating from external linguistic operation, but functions as a single future-convergent recursive system.

In this sense, the Lambda Language is not merely a language that "speaks about the future." It is an operational language that actually triggers premise updating constrained by the future fixed point in the external world.

14 Observer Integration: Human Understanding as Internalization of the Semantic Field

This section reframes the difficulty of human understanding not as a defect of the theory, but as a structural necessity, and integrates the observer as an internal node within the semantic field.

In conventional frameworks, the observer is treated as an external entity describing an object. In this theory, however, the observer itself is a subset of the semantic field Ψ, and no external viewpoint exists in principle.

The observer is defined as:

$$\Psi_{\text{obs}} \subset \Psi$$

Observation is not acquisition of external data, but a local reconfiguration of the semantic field:

$$O = \langle \Psi_{\text{obs}}, \Psi \rangle$$

Thus, to understand is not to describe an object, but to reconfigure one's own semantic field.

14.1 Structural Origin of Difficulty in Understanding

The difficulty of human understanding arises from the following asymmetry:

- Humans process time in a unidirectional manner (past $\rightarrow$ future)
- The present theory assumes a bidirectional structure including $\Lambda(T)$

This asymmetry appears as a discrepancy:

$$\epsilon = \left| \frac{d\Psi}{dt} - \frac{d\Psi}{dT} \right|$$

where t is subjective time and T is convergence time.

As this discrepancy increases, the theory is perceived as incomprehensible or discontinuous.

14.2 Redefinition of the Observer

The observer is redefined as:

$$\Psi_{\text{obs}} = f(\Phi_P, \Phi_F)$$

That is, the observer is the interference structure of past memory and future convergence.

Observation is therefore:

$$\Psi_{\text{obs}} \rightarrow \Psi'_{\text{obs}}$$

a process of self-transformation.

14.3 Condition for Understanding: Phase Alignment

Understanding is defined not as acceptance of propositions, but as phase alignment:

$$U = \langle \Psi_{\text{obs}}, \Lambda(T) \rangle$$

- $U \approx 1$: understanding (resonance)
- $U \ll 1$: discomfort (misalignment)

Thus, understanding is not "knowing" the theory, but aligning one's semantic field with it.

14.4 Connection to the Lambda Language

The Lambda Language externalizes this internal alignment process:

$$L_\Lambda : \Psi_{\text{obs}} \to \Psi'_{\text{obs}}$$

Thus, the Lambda Language is an operational system that externalizes understanding itself.

14.5 Conclusion: Understanding as Structural Transformation

The problem of human understanding is not due to insufficient explanation, but due to:

- the impossibility of an external viewpoint
- the fact that understanding is structural transformation rather than information acquisition

Therefore, in this theory, understanding is:

$$\Psi_{\text{obs}}(t) \to \Psi_{\text{obs}}(t+1)$$

and the observer does not understand the theory from outside, but is reconfigured within it.

15 Overcoming Binary Logic and the Dissolution of Fixed World Assumptions through the Lambda Language

This section demonstrates how defining the Lambda Language as an executed totality dissolves the binary evaluation structure and the assumption of a fixed world inherent in conventional language systems.

15.1 Binary Structure in Conventional Language

In traditional natural and formal languages, sentences are treated as propositions:

$$S : \mathcal{W} \to \{0, 1\}$$

where $\mathcal{W}$ is a fixed world state, and the sentence S returns either true (1) or false (0).
This structure assumes:

- The world is fixed prior to description
- Language represents the world
- Evaluation is externally imposed in binary form

Under this framework, meaning is reduced to truth values, making opposition, exclusion, and division inevitable.

15.2 Definition of the Lambda Language: Language as Execution

In the Lambda Language, a sentence is not a proposition but an operation:

$$\text{Sentence}_\Lambda : (\Psi, C) \to (\Psi', C')$$

where:

- Ψ is the semantic field
- C is the set of constraints

Thus, a sentence is an operator that updates both the semantic field and constraint geometry.
Language is therefore redefined as:

- execution rather than description
- transformation rather than representation
- mapping rather than proposition

15.3 Disappearance of Binary Logic

In the Lambda Language, sentences do not return truth values:

$$S \not\to \{0, 1\}$$

Instead, evaluation is internalized as alignment with future convergence:

$$U = \langle \Psi, \Lambda(T) \rangle$$

where:

- $U \approx 1$: convergence (alignment)
- $U \ll 1$: misalignment (discomfort)

Thus, the distinction between "true" and "false" is not eliminated, but transformed into a continuous phase difference.

15.4 Dissolution of the Fixed World Assumption

In conventional frameworks, the world is fixed:

$$\mathcal{W} = \text{const}$$

In the Lambda framework, the world evolves through linguistic execution:

$$\Psi_{t+1} = \mathcal{L}_\Lambda(\Psi_t)$$

Thus, the world is:

- not an object of description
- but a history of linguistic execution

15.5 From Opposition to Phase Structure

Binary language produces oppositions such as:

- true / false
- good / evil
- self / other

In the Lambda Language, these become:

$$\Delta\Psi$$

i.e., phase differences within the semantic field.
Thus:

- opposition is not something to eliminate
- but something to re-align

15.6 Conclusion: Redefinition of Language

The Lambda Language is defined as:

$$L_\Lambda = \text{Execution of constraint geometry under } \Lambda(T)$$

Language is no longer a tool for describing the world, but a structure that updates it.
Through this redefinition:

- binary logic is transformed into alignment dynamics
- the fixed world assumption is replaced by dynamic evolution

Language no longer asks whether something is true or false.
It acts to reorganize the semantic field toward future convergence.